

AYUDA MI CÓDIGO EN C NO FUNCIONA Y NO SÉ QUÉ HACER

Una guía paso a paso para solucionar tus problemas

Martín Illanes

2023-2

Esta es una guía relativamente nueva. Cualquier duda, reclamo, error o sugerencia que encuentres en esta guía por favor avisa a través de un Issue en este repositorio.

¡Hola! Soy un ayudante de Estructura de Datos y Algoritmos y te doy la bienvenida a esta guía titulada “*AYUDA MI CÓDIGO EN C NO FUNCIONA Y NO SÉ QUÉ HACER: una guía paso a paso para solucionar tus problemas.*”

En primer lugar, te pido calma, a todo el mundo le falla el código. A mí me pasó varias veces y aquí estoy, siendo ayudante del curso. Además, estás en el lugar indicado para resolver tu problema.

En esta guía te iré haciendo algunas preguntas para saber cómo ayudarte. ¡Te deseo suerte!

Índice

1. Primera pregunta: ¿Tu código compila?	2
1.1. Primero que todo, pregúntate, ¿hiciste todo el setup correctamente?	2
1.2. ¿Te funciona <code>make</code> ?	2
1.3. <code>make</code> me tira un error raro que parte con <code>src/</code> y tiene colores	3
1.4. ¿Estás ejecutando correctamente?	3
2. ¿Tienes <i>StackOverflow</i>, <i>SegmentationFault</i>, <i>Invalid Read</i>, etc?	4
2.1. ¿Did you tried Valgrind?	5
3. ¿Aún no lo puedes resolver?	7
4. ¿¿Aún no lo puedes resolver!!?	7
5. ¿Te rendiste?	7
6. Anexo: “Mi código está perfecto, pero el servidor lo corrige mal ¿Qué hago?”	8
6.1. ¿100 % de seguridad que tu respuesta está correcta?	8
6.2. ¿Generas un archivo con un output?	8
6.3. ¿Tienes un <code>.gitignore</code> ?	8
6.4. ¿Modificaste el output?	8

1. Primera pregunta: ¿Tu código compila?

Primero que todo, te pregunto. ¿Tu código compila? Con esto me refiero a que `make` corre correctamente.

Si se ejecuta el código (o parte de él) y se muestra algún error de C (StackOverflow, SegmentationFault, Invalid Read, etc), entonces el programa compila. *En caso de ser así: pasa a la segunda pregunta. Si no, continuemos.*

Tu código no corre

Ya, tu código ni siquiera corre. Espero que no se entregue hoy día la tarea.

1.1. Primero que todo, pregúntate, ¿hiciste todo el setup correctamente?

Prueba ingresando a la consola los siguientes comandos:

- Si usas Mac, ingresa: `xcode-select -version`
- Si usas Windows, ingresa: `wsl -l -v`

Si no te aparecen las versiones, preocúpate, es probable que no hayas hecho el setup correctamente. Contáctate con un ayudante o levanta una issue si necesitas más ayuda.

NOTA: Si usas Linux, C y sus herramientas son parte del sistema operativo. Por lo que es muy difícil que tu problema esté aquí.

1.2. ¿Te funciona make?

En el repositorio de tu tarea, ejecuta en la consola `make`

¿Qué te salió? Busca tu output:

- `done compiling`
¡FELICIDADES! Todo está bien (por ahora). Pasa al 1.3.
- `make : El término 'make' no se reconoce como nombre de un cmdlet, función, archivo de script o programa ejecutable.`

Esto significa que no estás en Linux. Intenta abriendo Linux y haciendo lo mismo.

NOTA: Si aquello te pasó en VS Code, puedes instalar la extensión WSL, apretar un símbolo similar a `>>` ubicado en la esquina inferior izquierda y luego seleccionar “conectar a WSL” (o similar) para abrir la consola de Linux en VS Code.

- `Command 'make' not found, but can be installed with:`

Hazle caso a tu PC e ingresa esos comandos.

NOTA: Si en algún punto la consola te dice `permision denied`, prueba poniendo `sudo` antes del comando que quieras.

NOTA 2: Esto es como mucho, pero te lo menciono igual. Te pueden aparecer más errores, pero hazle caso a la consola nomás. Por ejemplo, te puede aparecer `E: Unable to fetch some archives, maybe run apt-get update or try with -fix-missing?`. En ese caso, corre `apt-get update` (tal como dice ahí).

- `make: *** No targets specified and no makefile found. Stop.`

Eso significa que en tu carpeta no hay un Makefile. Intenta abriendo el repositorio del proyecto (y que la consola esté situada ahí).

- `/bin/sh: 1: gcc: not found`

Eso significa que no has instalado gcc. Escribe gcc en la consola y lee dos puntos más arriba en esta guía.

- `wrong exec format o cannot execute binary file: Exec format error`

Ejecutando `make clean` se debiese solucionar.

- ¿Ninguna de las anteriores?!? Te recomendamos [levantar una issue](#) o [contactarte con un ayudante](#).

1.3. make me tira un error raro que parte con src/ y tiene colores

Por ejemplo, algo así:

```
src/ratatouille/main.c: In function 'main':
src/ratatouille/main.c:63:3: error: expected ',', or ';' before 'FILE'
 63 |     FILE *output_file = fopen(argv[2], "w");
    |     ~~~~
src/ratatouille/main.c:179:17: error: 'output_file' undeclared (first use in this
      function); did you mean 'input_file'?
179 |         fprintf(output_file, "TABLE %i:%i FULL\n", location_id, table_id);
    |         ~~~~~~
    |         input_file
src/ratatouille/main.c:179:17: note: each undeclared identifier is reported only once for
      each function it appears in
make: *** [Makefile:111: obj/ratatouille/main.o] Error 1
```

En este caso, cuando `make` te dice `src/ratatouille/main.c:63:3:`, te está explicitando que la compilación falló en el archivo `main.c`, ubicado en `src/ratatouille/`, específicamente en la línea 63 y carácter 3.

En este caso, tal como dice `make`, en la línea anterior a aquella faltaba un `;`.

NOTA: En C, muchos errores pueden suceder en líneas anteriores y crear “errores de arrastre” en el resto del código. Te recomiendo poner atención a la sintaxis de tu código e ir resolviendo error por error. En este caso en específico, en la línea 62 me faltó un `;`, lo que provocó un error en la línea 63, haciendo que no se declarara correctamente el `output_file`. Esto provocó que la línea 179 también fallase.

1.4. ¿Estás ejecutando correctamente?

Recuerda que en las tareas aparece el formato con que se debe ejecutar la tarea y este sigue siempre un estilo así `./dcchuek input.txt output.txt`. Donde lo que sigue `./` es el nombre del ejecutable e `input.txt` y `output.txt` son los PATHS de los respectivos archivos (desde donde los estás ejecutando).

NOTA: Recuerda hacer siempre `make` antes de ejecutar tu programa.

Consideraciones especiales

Nosotros recomendamos fuertemente que sigas las guías del setup al pie de la letra y el uso de VS Code, ya que la gran mayoría de los ayudantes utilizamos también VS Code y te podremos ayudar de manera más fácil.

Si no te gusta VS Code y, por ejemplo, eres del Maule, te gusta usar *CuriCODE* y tienes problema con el setup; puedes preguntarnos de todas maneras, pero no te garantizamos que te podamos ayudar.

2. ¿Tienes *StackOverflow*, *SegmentationFault*, *Invalid Read*, etc?

Eso puede ser cualquier cosa.

¿Te acuerdas que en **python** existían errores que te informaban cuál era el problema (*EOFError*, *ImportError*, *IndexError*, *KeyError*, *KeyboardInterrupt*, *MemoryError*, *NameError*, *RuntimeError*, *StopIteration*, *SyntaxError*, *IndentationError*, *ValueError*, etc)? **Lamentablemente C no es así.**

En simples palabras, un *SegmentationFault* indica que, durante la ejecución del programa, C se metió en un segmento de memoria donde no se tuvo que haber metido. ¿Por qué ocurrió eso? No tengo ni la más mínima idea. Pudo haber sido cualquier cosa.

Es como si nos dijeras “¿Qué hacer si mi auto no anda? Dame una respuesta ahora que debo ir a entregar mi tarea en 5 minutos”. Existen muchos motivos por los cuales tu “auto” podría no funcionar. Probablemente nuestra respuesta sería “Abre el capó y échale una mirada, mira si encuentras algo y nos avisas”.

Algunos ejemplos de código que tira *SegmentationFault*

- Modificar strings

```
int main(void)
{
    char *s = "hello_world";
    *s = 'H';
}
```

- Referenciar un NULL

```
int *ptr = NULL;
printf("%d", *ptr);
```

- Acceder a una dirección ahora libre

```
int* p = (int*)malloc(8);
*p = 100;

free(p);

*p = 110;
printf("%d", *p);
```

- Array fuera de índice

```
int arr[2];  
arr[3] = 10;
```

- Cosas como estas (mal uso de `scanf()`)

```
int n = 2;  
scanf("%d", n);
```

¡Y muchas otras formas más! En todos estos casos, directa o indirectamente se accede mal a la memoria (aquello puede tomar las más variadas formas en tu tarea de más de 100 líneas de código).

Algunos ejemplos de código que tira *StackOverflow*

- Recursiones infinitas (*o muy largas*)

```
int foo()  
{  
    return foo();  
}
```

- Variables muy largas (*un array de 1048576 doubles, ¿qué podría salir mal?*)

```
double x[1048576];
```

2.1. ¿Did you tried Valgrind?

En la guía de setup que leíste atentamente y seguiste al pie de la letra, pudiste evidenciar que Valgrind se instala con el siguiente comando `sudo apt install valgrind`.

Cuando lo tengas instalado, ejecuta

```
valgrind --leak-check=full --show-leak-kinds=all --track-origins=yes --log-file=valgrind-  
out.txt ./executable input.txt output.txt
```

NOTA: Recuerda reemplazar `./executable input.txt output.txt` por cómo sea que se ejecuta la tarea que estás haciendo.

¿Valgrind te va a solucionar todos tus problemas? No. Pero te va a decir dónde hay *memory leaks* (usará un formato muy similar al ocupado en el 1.3). Normalmente, solucionando los memory leaks puedes solucionar tu *StackOverflow*, *SegmentationFault*, *Invalid Read*, *etc*; por lo que no mires en menos a esta gran herramienta.

¿Cómo leer el *output* de Valgrind?

Si usas el comando mostrado anteriormente, se te mostrará tanto en la consola como en un archivo `valgrind-out.txt` el output de Valgrind.

Este tendrá un formato similar al siguiente:

```
==13097== Memcheck, a memory error detector
==13097== Copyright (C) 2002-2017, and GNU GPL'd, by Julian Seward et al.
==13097== Using Valgrind-3.15.0-608cb11914-20190413 and LibVEX; rerun with -h for
    copyright info
==13097== Command: ./EJECUTABLE INPUT.txt OUTPUT.txt
```

Primero, habrá una especie de “header” con información sobre el entorno de ejecución y el llamado de Valgrind. Para propósitos de esta guía, ignoraremos ese output, ya que normalmente los problemas se pueden solucionar de manera más simple.

En algún minuto, llegaremos a un error de formato similar a este:

```
==13097== Use of uninitialised value of size 8
==13097==    at 0x109231: insertar_matriz (hashtable.c:6)
==13097==    by 0x109F08: main (main.c:182)
==13097== Uninitialised value was created by a heap allocation
==13097==    at 0x483B7F3: malloc (in /usr/lib/x86_64-linux-gnu/valgrind/
    vgpreload_memcheck-amd64-linux.so)
==13097==    by 0x109533: submatrices (main.c:30)
==13097==    by 0x109EDE: main (main.c:181)
```

Este error, por ejemplo, me dice que, una función en la línea 182 de `main.c`, creó una variable sin utilizar en la línea 6 de `hashtable.c` (función `insertar_matriz`).

También pueden haber errores así:

```
==13097== Conditional jump or move depends on uninitialised value(s)
==13097==    at 0x109237: insertar_matriz (hashtable.c:6)
==13097==    by 0x109F08: main (main.c:182)
==13097== Uninitialised value was created by a heap allocation
==13097==    at 0x483B7F3: malloc (in /usr/lib/x86_64-linux-gnu/valgrind/
    vgpreload_memcheck-amd64-linux.so)
==13097==    by 0x109D29: main (main.c:142)
```

Este error, por ejemplo, me dice que, una función en la línea 182 de `main.c`, tiene en la línea 6 de `hashtable.c` (función `insertar_matriz`) un salto condicional (*if, else o similares*) que depende de un valor que podría no estar presente.

Uso de memoria:

```
==13097== HEAP SUMMARY:
==13097==    in use at exit: 179,187,172 bytes in 1,620,624 blocks
==13097==    total heap usage: 1,620,627 allocs, 3 frees, 179,199,460 bytes allocated
```

Luego de los errores, se mostrará el total de memoria usada en total. **NOTA:** *Recuerda que hay tareas que tienen límites de uso de memoria.*

Memoria sin liberar o perdida:

```
==13097== Searching for pointers to 1,620,624 not-freed blocks
==13097== Checked 75,336 bytes
==13097==
==13097== 76 bytes in 1 blocks are possibly lost in loss record 1 of 18
```

```
==13097==    at 0x483B7F3: malloc (in /usr/lib/x86_64-linux-gnu/valgrind/
vgpreload_memcheck-amd64-linux.so)
==13097==    by 0x109533: submatrices (main.c:30)
==13097==    by 0x109EDE: main (main.c:181)
```

También, se mostrarán bloques de memoria sin liberar. Acá se muestra que se hizo un `malloc` en la función `submatrices` llamada en la línea 30 de `main.c` y que no ha sido liberado.

Resumen de problemas de memoria:

```
==13097== LEAK SUMMARY:
==13097==    definitely lost: 7,319,008 bytes in 4,659 blocks
==13097==    indirectly lost: 171,866,516 bytes in 1,615,960 blocks
==13097==    possibly lost: 232 bytes in 2 blocks
==13097==    still reachable: 1,416 bytes in 3 blocks
==13097==    suppressed: 0 bytes in 0 blocks
==13097==
==13097== ERROR SUMMARY: 1665730 errors from 16 contexts (suppressed: 0 from 0)
==13097==
```

Cerca del final del output de *Valgrind* se mostrará un **LEAK SUMMARY**, el cual indicará la cantidad de memoria perdida y/o sin liberar y el total de errores de memoria encontrados.

Es buena práctica no tener errores de memoria y, generalmente, el no tenerlos está asociado a que tu tarea funcione. **NOTA:** Además, en algunas tareas, se puede dar un puntaje BONUS por no tener problemas de memoria.

3. ¿Aún no lo puedes resolver?

Te recomendamos que uses la **Guía para debuggear en C**.

Muchas veces la sintaxis del código está correcta, pero el código en general es el que falla.

¿No entiendes a lo que me refiero? Es como escribir “*¡Oye C! Por favor, échale jugo de naranja al motor del auto.*” Está bien escrita la instrucción, por lo que alguien revisando la gramática y ortografía no encuentra ningún error. De todos modos, la instrucción hace fallar al programa.

¡Para esos casos **debuggear** sirve mucho! Te permite ir paso a paso viendo cómo cambian las variables y pillar bien el error que hace que todo falle.

4. ¿¿Aún no lo puedes resolver?!?

No pasa nada. Yo en lo personal me llegué a gastar 3 cupones (72h de atraso) pillando un puntero mal puesto. ¡Te deseo ánimo y paciencia!

5. ¿Te rendiste?

Recuerda que siempre puedes [levantar una issue](#) o [contactarte con un ayudante](#).

6. Anexo: “Mi código está perfecto, pero el servidor lo corrige mal ¿Qué hago?”

6.1. ¿100 % de seguridad que tu respuesta está correcta?

¿Probaste utilizar herramientas como el [comando diff](#) o [diffchecker](#)? Estas herramientas permiten comparar el output generado con el output de la pauta y es lo que el servidor utiliza para poder asegurarse de que la respuesta esté correcta.

6.2. ¿Generas un archivo con un output?

Muchas veces se ocupa `printf()` en vez de `fprintf()` (sólo se imprime a consola y no a archivo). Como los resultados no se imprimen a archivo, el output no es correcto y se genera el *Wrong Output*.

6.3. ¿Tienes un .gitignore?

Sucede que a veces los `.gitignore` están mal configurados, por lo que se podrían excluir archivos esenciales para el desarrollo de la tarea. Te recomendamos excluir solamente elementos ajenos al código de la tarea, como lo pueden ser el enunciado en PDF o los testcases.

6.4. ¿Modificaste el output?

Puede pasar que para propósitos de debuggeo se modifica el output o se hardcodea la dirección del *output file*. Esto puede generar problemas con el servidor, ya que este incluye sus propios parámetros en el nombre de archivo.

Creado por [Martín Illanes](#) el 2023-2